

- [graph.py — Deep Function Reference with Code](#)
 - [Legend](#)
- [Part 1: Module-Level Utilities](#)
 - [_parse_planner_specialists\(content: str\) -> list\[str\]](#)
 - The actual code:
 - Why two passes?
 - [_label\(stage: str\) -> str](#)
 - [_desc\(stage: str\) -> str](#)
- [Part 2: build_graph — Main Analysis Builder](#)
 - [_progress\(stage, msg\)](#)
 - [truncate\(text, limit\)](#)
 - [normalize_content\(content\)](#)
 - [compress_messages\(messages\)](#)
 - Phase 1: Window capping (line 262)
 - Phase 2: Per-message truncation (lines 265–284)
 - [strip_tool_messages\(messages\)](#)
 - What it does:
 - [_extract_structured_output\(content\)](#)
 - [_stream_structured_outputs\(owner_name, output_str\)](#)
 - The double-parse problem:
 - Streaming mechanics:
 - [build_focused_context\(all_msgs, stage_name\)](#)
 - Step 1: Identify my tool call IDs
 - Step 2: The categorization engine
 - Step 3: Assembly
 - [make_specialist_node\(name\) → specialist_node](#)
 - What the closure captures:
 - The returned node function:
 - [planner_node\(state\)](#)
 - Three input paths:
 - [_heuristic_check\(messages, stage_name\)](#)
 - How the scan works:
 - What it looks for in `execute_python` outputs:
 - Decision tree:
 - [_maybe_extend_pipeline\(state\)](#)
 - Mechanics:
 - [manager_review_node\(state\)](#)
 - Decision hierarchy (in order):
 - [tool_node\(state\)](#)

- Key details:
- `specialist_router(state)`
- `after_tools(state)`
 - How it works:
- `_parse_extensions(messages, stages)`
 - Parsing rules:
- `manager_router(state)`
 - Three paths:
- `make_stage_entry(stage_name) → entry_node`
 - Special case for critic:
- Graph Assembly (lines 869–924)
 - Key design decisions:
- Part 3: `build_followup_graph` — Follow-Up Builder
 - `_split_leading_system(msgs)`
 - `followup_router_node(state)`
 - Parsing the LLM response:
 - The safety net (lines 1084–1115):
 - Why store a marker?
 - `followup_executor_node(state)`
 - Three branches:
 - `followup_tool_node(state)`
 - `executor_router(state)`
 - Follow-Up Graph Assembly (lines 1223–1238)
- Part 4: Logic Flows
 - Flow 1: New Question → Sequential Analysis
 - Flow 2: New Question → Parallel Analysis
 - Flow 3: Manager Rework Loop
 - Flow 4: Critic Pipeline Extension
 - Flow 5: Follow-Up Question
 - Flow 6: Heuristic Auto-Accept (Happy Path)
- Part 5: Configuration Overrides
 - Settings table
 - Override path through the stack:

graph.py — Deep Function Reference with Code

File: `python/agentik/graph.py`

Scope: Every callable with actual code excerpts, deep mechanical explanations, and

Legend

Tag	Meaning
Node	LangGraph node function — runs as part of the state machine and returns state updates.
Helper	Pure utility used inside nodes; no LangGraph contract.
Router	Conditional edge function — returns a string key that LangGraph maps to the next node.
Factory	Returns another function (used to create nodes dynamically with different prompts/tools).
Callback	External function injected into the graph (e.g. <code>on_progress</code>).

Part 1: Module-Level Utilities

```
_parse_planner_specialists(content:  
str) -> list[str]
```

Type: Helper | **Lines:** 97–139

This is the **parser that keeps the pipeline alive**. The planner is asked to emit a JSON object telling us which specialists to run, but LLMs routinely wrap JSON in markdown fences, add prose, or miss commas. Without this parser, a single malformed planner response would kill the entire analysis.

The actual code:

```
def _parse_planner_specialists(content: str) -> list[str]:  
    import json as _json  
    import re as _re  
  
    def _norm(name: str) -> str:  
        return name.strip().lower().replace(" ", "_")
```

```

selected: list[str] = []
text = (content or "").strip()

# 1) Try JSON – tolerate ```json fences / surrounding prose
match = _re.search(r"\{.*\}", text, _re.DOTALL)
if match:
    try:
        obj = _json.loads(match.group(0))
        raw = obj.get("specialists") or obj.get("agents") or []
        if isinstance(raw, str):
            raw = _re.split(r"[,\n]", raw)
        for s in raw:
            ns = _norm(str(s))
            if ns in SPECIALIST_POOL and ns not in selected:
                selected.append(ns)
    except (_json.JSONDecodeError, ValueError, AttributeError):
        pass

# 2) Fallback: legacy "SPECIALISTS: a, b" line
if not selected:
    for line in text.split("\n"):
        line = line.strip()
        if line.upper().startswith("SPECIALISTS:"):
            for s in line.split(":", 1)[1].split(","):
                ns = _norm(s)
                if ns in SPECIALIST_POOL and ns not in selected:
                    selected.append(ns)

return selected

```

Why two passes?

Pass 1 (JSON): The preferred format. The regex `r"\{.*\}"` with `re.DOTALL` greedily finds the first `{...}` block anywhere in the text. This means the LLM can write prose before and after the JSON, and we still grab the right object. We accept both `"specialists"` and `"agents"` keys because different prompt versions use different terminology. If the value is a string instead of a list (another common LLM mistake), we split on commas or newlines.

Pass 2 (Legacy line): If JSON parsing yields nothing, we fall back to a simpler contract: a line starting with `SPECIALISTS:`. This is a last resort for models that ignore the JSON instruction entirely.

Validation: Every candidate is normalized (`_norm`) and checked against `SPECIALIST_POOL`:

```

SPECIALIST_POOL = [
    "analyst", "forecaster", "anomaly_detective",
    "bayesian_analyst", "optimizer", "shift_reporter",
]

```

Only names in this list are accepted. Everything else is silently dropped. This prevents a hallucinated specialist name from breaking downstream nodes.

Deduplication: The `ns not in selected` guard ensures each specialist appears only once, even if the LLM repeats itself.

`_label(stage: str) -> str`

Type: Helper | **Lines:** 148–149

```
_STAGE_LABELS: dict[str, str] = {
    "data_loader": "Зареждане на данни",
    "planner": "Планиране",
    "analyst": "Анализатор",
    "forecaster": "Прогнозиране",
    "anomaly_detective": "Детектор на аномалии",
    "bayesian_analyst": "Байесов анализ",
    "optimizer": "Оптимизатор",
    "shift_reporter": "Сменен отчет",
    "critic": "Проверка и валидация",
    "reporter": "Генериране на отчет",
    "manager": "Мениджър",
}

def _label(stage: str) -> str:
    return _STAGE_LABELS.get(stage, stage)
```

A trivial lookup, but it centralizes the UI-facing Bulgarian strings. If a new stage is added, only `_STAGE_LABELS` needs updating. Every progress message sent to the frontend goes through `_label` so the user sees "Анализатор" instead of "analyst".

`_desc(stage: str) -> str`

Type: Helper | **Lines:** 163–164

```
_STAGE_DESCRIPTIONS: dict[str, str] = {
    "data_loader": "Зареждане на данни от базата...",
    "analyst": "Статистически анализ, разпределения и SPC диаграми...",
    "forecaster": "Прогнозиране на трендове и сезонност...",
    "anomaly_detective": "Търсене на аномалии и причини...",
    "bayesian_analyst": "Байесов анализ и доверителни интервали...",
    "optimizer": "Оптимизация на настройки и препоръки...",
    "shift_reporter": "Анализ по смени и KPI показатели...",
    "critic": "Валидация на резултати и крос-проверка на числата...",
}
```

```
"reporter": "Писане на краен отчет...",
}

def _desc(stage: str) -> str:
    return _STAGE_DESCRIPTIONS.get(stage, "")
```

Used alongside `_label` for the **first iteration** of a specialist. The UI shows something like:

Анализатор: Статистически анализ, разпределения и SPC диаграми...

Subsequent iterations are silent to avoid UI noise.

Part 2: `build_graph` — Main Analysis Builder

`build_graph` (lines 182–924) is the outer function. Everything inside it is a closure over shared state (`llm`, `tools_by_name`, settings, etc.).

`_progress(stage, msg)`

Type: Callback wrapper | **Lines:** 192–193

```
_progress = on_progress or (lambda stage, msg: None)
```

This is a **closure** created at graph build time. If the caller (e.g. `api_endpoint.py`) provides an `on_progress` callback, every node uses `_progress` to stream live updates. If not, it is a no-op lambda that silently discards everything.

Why a closure instead of a global? Because `build_graph` is called per-request with a different callback each time. A global would leak progress events between concurrent analyses.

Usage pattern inside nodes:

```
_progress("planner", "Планиране: Избор на подходящи специалисти...")
```

The frontend polls the analysis status endpoint, and the callback writes these messages into a shared progress store.

truncate(text, limit)

Type: Helper | **Lines:** 252–253

```
def truncate(text: str, limit: int) -> str:
    return text[:limit] + "\n... [truncated]" if len(text) > limit else text
```

A brutally simple text slicer. It preserves the **beginning** of the text (the most important part) and appends a truncation marker. Used everywhere message history is compressed.

Important detail: The `\n` before `... [truncated]` ensures the marker starts on its own line, making it easy to spot in logs.

Usage:

- Tool outputs truncated to `_MAX_TOOL_OUTPUT_CHARS` (default 2000)
- AI messages truncated to `_MAX_AI_MSG_CHARS` (default 3000)
- Prior summaries truncated to various caps (200, 400, 1200, 1500)

normalize_content(content)

Type: Helper | **Lines:** 255–259

```
def normalize_content(content) -> str:
    if isinstance(content, list):
        texts = [
            item.get("text", "") if isinstance(item, dict) else str(item)
            for item in content
        ]
        return "\n".join(texts).strip()
    return str(content) if content else ""
```

The Gemini-formatting problem: Google's Gemini API sometimes returns `response.content` not as a string, but as a list of dicts:

```
[{"text": "Here is the analysis"}, {"text": "of your mill data"}]
```

Other times it returns a plain string. If we try to call `.upper()` or `.startswith()` on a list, Python throws `AttributeError`. This function normalizes both shapes into a single string.

Algorithm:

1. If `content` is a list → iterate each item.
 - If item is a dict, extract `"text"`.
 - Otherwise, coerce to string.
 - Join all pieces with newlines.
2. Otherwise → `str(content)` or `""` if falsy.

This is the first function called on every LLM response before any further parsing happens. Without it, the entire graph would crash on Gemini's list-of-dicts format.

compress_messages(messages)

Type: Helper | **Lines:** 261–285

```
def compress_messages(messages: list[BaseMessage]) -> list[BaseMessage]:
    if len(messages) > _MAX_MESSAGES_WINDOW + 1:
        messages = [messages[0]] + messages[-(_MAX_MESSAGES_WINDOW):]

    compressed = []
    for msg in messages:
        if isinstance(msg, ToolMessage):
            content = normalize_content(msg.content)
            compressed.append(ToolMessage(
                content=truncate(content, _MAX_TOOL_OUTPUT_CHARS),
                tool_call_id=msg.tool_call_id, name=msg.name,
            ))
        elif isinstance(msg, AIMessage):
            content = normalize_content(msg.content)
            if len(content) > _MAX_AI_MSG_CHARS:
                compressed.append(AIMessage(
                    content=truncate(content, _MAX_AI_MSG_CHARS),
                    name=getattr(msg, "name", None),
                    tool_calls=msg.tool_calls if msg.tool_calls else [],
                ))
            else:
                compressed.append(msg)
        else:
            compressed.append(msg)
    return compressed
```

Two-phase compressor designed to prevent token explosions in long conversations.

Phase 1: Window capping (line 262)

```
if len(messages) > _MAX_MESSAGES_WINDOW + 1:
    messages = [messages[0]] + messages[-(_MAX_MESSAGES_WINDOW):]
```

This is a **sliding window** that always preserves:

- `messages[0]` — typically the original user question or the first system prompt. This is sacred; without it the LLM loses the task context.
- The last `_MAX_MESSAGES_WINDOW` messages (default 14) — the most recent context.

Everything in between is discarded. In a 10-stage pipeline where each stage produces 3-5 messages, this cap is the difference between staying within context limits and exceeding them.

Phase 2: Per-message truncation (lines 265–284)

Even within the window, individual messages can be enormous (e.g. a Python stack trace or a large DataFrame summary). This phase rebuilds oversized messages:

- **ToolMessage** → Reconstructed with truncated content. The `tool_call_id` and `name` are preserved because LangGraph needs them for routing.
- **AIMessage** → If content exceeds cap, rebuilt with truncated text. **Crucially**, `tool_calls` are preserved even when truncating, because the next routing step depends on whether `tool_calls` exist.
- **Other messages** (HumanMessage, SystemMessage) → Passed through unchanged. These are usually small and important.

Immutability note: The function returns a *new* list of message objects. It does not mutate the original list. This matters because LangGraph may checkpoint the original state.

`strip_tool_messages(messages)`

Type: Helper | **Lines:** 287–301

```
def strip_tool_messages(messages: list[BaseMessage]) -> list[BaseMessage]:
    clean = []
    for msg in messages:
        if isinstance(msg, ToolMessage):
```

```

        content = normalize_content(msg.content)
        clean.append(AIMessage(
            content=f"[Tool result from {msg.name}]: {truncate(content, 800)}",
            name=msg.name,
        ))
    elif isinstance(msg, AIMessage) and msg.tool_calls:
        text = normalize_content(msg.content) or f"[{getattr(msg, 'name', 'agent')}
requested tools]"
        clean.append(AIMessage(content=text, name=getattr(msg, "name", None)))
    else:
        clean.append(msg)
return clean

```

The provider-rejection problem: Some LLM providers (including certain Gemini configurations) reject `ToolMessages` when sent to an LLM that does not have tools bound. The `planner_node` and `manager_review_node` are pure reasoning nodes with no tools attached, so sending raw `ToolMessage` objects to them causes API errors.

What it does:

1. `ToolMessage` → `AIMessage` with a narrative prefix: `[Tool result from execute_python]: ....` This preserves the semantic content while changing the object type.
2. `AIMessage` with `tool_calls` → Text-only `AIMessage`. The tool request itself is converted to narrative form (e.g. "analyst requested tools").
3. **Everything else** → Passed through.

Why 800 chars for tool results? When stripping, we're sending to a non-tool LLM that just needs a *sense* of what happened. 800 chars is enough for a summary without overwhelming the context.

Usage: Called exclusively by `planner_node` and `manager_review_node` before their LLM invocations.

`_extract_structured_output(content)`

Type: Helper | **Lines:** 304–321

```

def _extract_structured_output(content: str) -> str | None:
    import json as _json
    lines = content.split("\n")
    structured_parts = []
    for line in lines:
        stripped = line.strip()

```

```

if stripped.startswith("STRUCTURED_OUTPUT:"):
    json_str = stripped[len("STRUCTURED_OUTPUT:").strip():].strip()
    try:
        parsed = _json.loads(json_str)
        structured_parts.append(_json.dumps(parsed, ensure_ascii=False))
    except _json.JSONDecodeError:
        pass
if structured_parts:
    return " | ".join(structured_parts)
return None

```

The 2C protocol: Skills can emit structured data by printing special lines:

```
print(f"STRUCTURED_OUTPUT:{_json.dumps({'metric': 'mean', 'value': 42.5})}")
```

This function scans the **raw stdout text** for those lines, parses the JSON, and returns a compact string. It is used in two contexts:

1. **Context building** — `build_focused_context` injects structured outputs into prior summaries so downstream agents get compact data instead of verbose stdout.
2. **Streaming** — `_stream_structured_outputs` (the next function) sends these to the frontend for live "data tiles".

Key design: It is lenient. Malformed JSON lines are silently skipped (`pass`), so a skill's typo does not break the pipeline. It also joins multiple structured blocks with " | " in case a skill prints several.

`_stream_structured_outputs(owner_name, output_str)`

Type: Helper | **Lines:** 329–363

```

def _stream_structured_outputs(owner_name: str, output_str: str) -> None:
    if not output_str or "STRUCTURED_OUTPUT:" not in output_str:
        return

    import json as _json
    candidate_text = output_str
    try:
        wrapper = _json.loads(output_str)
        if isinstance(wrapper, dict) and isinstance(wrapper.get("stdout"), str):
            candidate_text = wrapper["stdout"]
    except (_json.JSONDecodeError, ValueError):
        pass

```

```

for line in candidate_text.split("\n"):
    stripped = line.strip()
    if not stripped.startswith("STRUCTURED_OUTPUT:"):
        continue
    payload_raw = stripped[len("STRUCTURED_OUTPUT:").:].strip()
    try:
        parsed = _json.loads(payload_raw)
    except _json.JSONDecodeError:
        continue
    try:
        compact = _json.dumps(parsed, ensure_ascii=False, default=str)
        if len(compact) > 4000:
            compact = compact[:3997] + "..."
        _progress(owner_name, f"STRUCTURED:{compact}")
    except Exception:
        continue

```

Live streaming to the frontend. While `_extract_structured_output` is for internal context building, this function is for **user-facing progress events**.

The double-parse problem:

`output_str` comes from the `execute_python` tool. It can be in two shapes:

1. **Raw stdout** — the literal text the skill printed.
2. **JSON wrapper** — `{"stdout": "...", "stderr": "...", "result": ...}` (depending on tool implementation).

The function first tries to parse as JSON wrapper and extract `"stdout"`. If that fails, it falls back to treating the whole thing as raw text.

Streaming mechanics:

1. Skip if no `STRUCTURED_OUTPUT:` substring (fast path).
2. For each matching line, parse the JSON payload.
3. Re-serialize with `ensure_ascii=False` so Bulgarian text survives.
4. Cap to 4000 chars to prevent a huge metrics dict from flooding the progress stream.
5. Call `_progress(owner_name, f"STRUCTURED:{compact}")`.

Frontend routing: The UI watches for messages starting with `STRUCTURED:` and routes them to a dedicated "live data tiles" panel, separate from the normal text log.

build_focused_context(all_msgs, stage_name)

Type: Helper | **Lines:** 366–430

This is the **context compression engine** that makes long pipelines feasible. Instead of sending a specialist the entire message history (which could be 50+ messages after several stages), it constructs a minimal focused subset containing only what that specialist needs.

```
def build_focused_context(all_msgs: list[BaseMessage], stage_name: str) ->
list[BaseMessage]:
    user_msg = None
    prior_summary_parts = []
    current_stage_msgs = []

    # Step 1: Find tool calls I made so I can find my tool results
    my_tool_call_ids = set()
    for msg in all_msgs:
        if isinstance(msg, AIMessage) and getattr(msg, "name", None) == stage_name and
msg.tool_calls:
            for tc in msg.tool_calls:
                my_tool_call_ids.add(tc.get("id"))

    # Step 2: Categorize every message
    for msg in all_msgs:
        if isinstance(msg, HumanMessage) and user_msg is None:
            user_msg = msg
            continue

        msg_name = getattr(msg, "name", None)

        if msg_name == stage_name:
            current_stage_msgs.append(msg)
            continue

        if isinstance(msg, ToolMessage) and msg.tool_call_id in my_tool_call_ids:
            content = normalize_content(msg.content)
            current_stage_msgs.append(ToolMessage(
                content=truncate(content, _MAX_TOOL_OUTPUT_CHARS),
                tool_call_id=msg.tool_call_id, name=msg.name,
            ))
            continue

    # Summarize prior stages compactly
    if isinstance(msg, ToolMessage) and msg.name in ("query_mill_data",
"query_combined_data"):
        content = normalize_content(msg.content)
        if "loaded" in content.lower():
            prior_summary_parts.append(truncate(content, 200))
    elif isinstance(msg, ToolMessage) and msg.name == "execute_python":
        content = normalize_content(msg.content)
        structured = _extract_structured_output(content)
        if structured:
            prior_summary_parts.append(f"[structured data]: {structured}")
```

```

        else:
            prior_summary_parts.append(f"[python output]: {truncate(content, 1200)}")
        elif isinstance(msg, AIMessage) and msg_name and msg_name not in ("manager",
"planner") and not msg.tool_calls:
            content = normalize_content(msg.content)
            if content:
                cap = 1500 if msg_name == "critic" else 400
                prior_summary_parts.append(f"[{msg_name}]: {truncate(content, cap)}")
        elif msg_name == "manager" and isinstance(msg, AIMessage):
            content = normalize_content(msg.content)
            if "REWORK" in content and stage_name in content:
                current_stage_msgs.append(msg)

# Step 3: Assemble
result = []
if user_msg:
    result.append(user_msg)

if prior_summary_parts:
    summary = "[Prior analysis context]:\n" + "\n".join(prior_summary_parts[-8:])
    result.append(HumanMessage(content=summary))

result.extend(compress_messages(current_stage_msgs))
return result

```

Step 1: Identify my tool call IDs

A specialist may call tools multiple times across iterations. We scan the full history for `AIMessages` where `name == stage_name` and `tool_calls` is present. Each tool call has an `id`. We collect all these IDs into `my_tool_call_ids` so we can later find the corresponding `ToolMessage` results.

Step 2: The categorization engine

This is the heart of the function. It iterates **once** through all messages and routes each to one of three buckets:

Bucket A: `current_stage_msgs` (my own work)

- Messages where `name == stage_name` → my own previous outputs.
- `ToolMessages` where `tool_call_id in my_tool_call_ids` → the results of *my* tool calls.
- Manager `REWORK` messages mentioning my stage → so I know I was sent back.

Bucket B: `prior_summary_parts` (what happened before me)

- Data loading confirmations → `[loaded ...]` (200 chars)

- Python outputs from other specialists → `[python output]: ...` (1200 chars) or `[structured data]: ...` if structured blocks exist.
- Other specialists' prose → `[analyst]: ...` (400 chars). **Exception:** critic gets 1500 chars because its verification block is critical for the reporter.

Bucket C: Discarded

- Everything else is silently dropped. This includes tool results from other specialists, manager `ACCEPT` messages, and the planner's raw JSON output.

Step 3: Assembly

The final message list is built in order:

1. **Original user question** (if found) — this is the anchor for every specialist.
2. **Prior analysis context** — the last 8 summary parts joined into a single `HumanMessage`. This gives the specialist a narrative of what has been discovered so far without the full verbosity.
3. **My own messages** (compressed) — the specialist sees its own prior outputs and tool results so it can continue coherently.

Why last 8? Empirically, 8 summary parts is enough to capture the essence of prior work without bloating the context. The number is not configurable because it's a context-compression constant, not a user preference.

`make_specialist_node(name)` → `specialist_node`

Type: Factory | **Lines:** 449–501

```
def make_specialist_node(name: str):
    system_prompt = ALL_PROMPTS[name]
    stage_llm = specialist_llms[name]

    def specialist_node(state: AnalysisState) -> dict:
        iteration = sum(1 for m in state["messages"] if getattr(m, "name", None) == name)
+ 1
        print(f"\n [{name}] iteration {iteration}/{_MAX_SPECIALIST_ITERS} -
processing...")
        if iteration == 1:
            desc = _desc(name)
            _progress(name, f"[_label(name)]: {desc}" if desc else f"[_label(name)]...")
```

```

if iteration > _MAX_SPECIALIST_ITERS:
    print(f" [{name}] Iteration cap reached, advancing.")
    return {
        "messages": [AIMessage(
            content=f"[{name}] Done (iteration cap). Moving on.",
            name=name,
        )],
    }

raw_msgs = state["messages"]
focused = build_focused_context(raw_msgs, name)
messages = [SystemMessage(content=system_prompt)] + focused

print(f" [{name}] Context: {len(focused)} msgs, types: {[type(m).__name__ for m
in focused]}")

try:
    response = stage_llm.invoke(messages)
except Exception as e:
    error_str = str(e)
    print(f" [{name}] LLM error: {error_str[:200]}")
    return {
        "messages": [AIMessage(
            content=f"[{name}] Error: {error_str[:150]}. Moving on.",
            name=name,
        )],
    }

response.content = normalize_content(response.content)

if response.tool_calls:
    tool_names = [tc["name"] for tc in response.tool_calls]
    print(f" [{name}] Calling tools: {tool_names}")
else:
    preview = (response.content[:120] + "...") if response.content and
len(response.content) > 120 else response.content
    print(f" [{name}] Done: \"{preview}\"")
    _progress(name, f"✓ {_label(name)} завърши.")

response.name = name
return {"messages": [response]}

return specialist_node

```

Factory pattern: This function does not run a specialist itself. It **creates** a node function tailored to a specific specialist name. This avoids duplicating the same runtime logic 6 times (once per specialist).

What the closure captures:

- `system_prompt = ALL_PROMPTS[name]` — the Bulgarian/English prompt telling the specialist what its job is.
- `stage_llm = specialist_llms[name]` — the Gemini model **with tools already bound** for this stage.

The returned node function:

Iteration counting (line 454):

```
iteration = sum(1 for m in state["messages"] if getattr(m, "name", None) == name) + 1
```

This scans the entire message history counting how many times this specialist has already appeared. It counts the specialist's own `AIMessages`. Adding 1 gives the current iteration number.

First-iteration progress (lines 457–459): Only on `iteration == 1` does it send a progress message to the frontend. This avoids spamming the UI with "analyst running..." on every loop.

Iteration cap (lines 461–468): If `iteration > _MAX_SPECIALIST_ITERS` (default 5), the specialist is forced to emit a "Done (iteration cap)" message and exit. This is a hard guardrail against infinite tool-calling loops.

Context building (lines 470–472): Calls `build_focused_context` to get a minimal message list, then prepends the specialist's system prompt.

LLM invocation (lines 476–486): Calls `stage_llm.invoke(messages)`. The model may return plain text or a list of tool calls. On any exception (network error, rate limit, content filter), it catches the error, logs it, and returns an error message — **the graph continues running** instead of crashing.

Tool call detection (lines 490–496):

- If `response.tool_calls` exists, it logs which tools are being called.
- If not, it logs a preview of the text response and sends a completion progress message.

Name tagging (line 498):

```
response.name = name
```

This is critical. LangGraph's routing depends on message names. Without this tag, `after_tools` and `specialist_router` would not know which specialist produced the message.

planner_node(state)

Type: Node | Lines: 528–587

```
def planner_node(state: AnalysisState) -> dict:
    print("\n [planner] Analyzing request to determine specialists needed...")
    _progress("planner", "Планиране: Избор на подходящи специалисти...")

    # Template override: skip LLM planning if a template was selected
    if _template_id:
        from analysis_templates import get_template_specialists
        tpl_specialists = get_template_specialists(_template_id)
        if tpl_specialists:
            selected = [s for s in tpl_specialists if s in SPECIALIST_POOL]
            if selected:
                stages = FIXED_PREFIX + selected + FIXED_SUFFIX
                readable = ' → '.join(_label(s) for s in selected)
                print(f" [planner] Using template '{_template_id}': {' → '.join(stages)}")
                _progress("planner", f"Шаблон: {readable}")
                return {
                    "messages": [AIMessage(content=f"Using template '{_template_id}'.",
SPECIALISTS: {' → '.join(selected)}", name="planner")],
                    "stages_to_run": stages,
                    "current_stage": "planner",
                }

            compressed = compress_messages(state["messages"])
            planner_system = PLANNER_PROMPT
            if _memory_hint:
                planner_system = f"{PLANNER_PROMPT}\n\n---\n{_memory_hint}"
            messages = [SystemMessage(content=planner_system)] + strip_tool_messages(compressed)

            try:
                response = llm.invoke(messages)
            except Exception as e:
                print(f" [planner] Error: {str(e)[:150]}. Defaulting to analyst only.")
                return {
                    "messages": [AIMessage(content="SPECIALISTS: analyst\nRATIONALE: Fallback due
to planning error.", name="planner")],
                    "stages_to_run": ["data_loader", "planner", "analyst", "critic", "reporter"],
                    "current_stage": "planner",
                }

            content = normalize_content(response.content)
            print(f" [planner] Response: {content}")

            selected = _parse_planner_specialists(content)

            if not selected:
                selected = ["analyst"]
                print(" [planner] No valid specialists parsed, defaulting to analyst.")

            stages = FIXED_PREFIX + selected + FIXED_SUFFIX
            print(f" [planner] Pipeline: {' → '.join(stages)}")
            readable = ' → '.join(_label(s) for s in selected)
            _progress("planner", f"Избрани специалисти: {readable}")
```

```
return {
    "messages": [AIMessage(content=content, name="planner")],
    "stages_to_run": stages,
    "current_stage": "planner",
}
```

The brain of the pipeline. This is the only stage that decides the pipeline shape. Every other stage just executes; the planner decides *what* to execute.

Three input paths:

1. Template override (lines 533–547) If the frontend sent a `template_id`, the planner is **bypassed entirely**. It lazy-imports `get_template_specialists` (to avoid circular imports) and uses the template's hardcoded list. This is fast, deterministic, and avoids wasting an LLM call when the user already told us what kind of analysis they want.

2. Memory-enhanced planning (lines 549–554) If long-term memory found similar past analyses, `_memory_hint` is injected into the planner's system prompt as soft guidance. The planner is told to use these as hints, not to copy them blindly.

3. Standard LLM planning (lines 557–587)

- Compresses the message history.
- Strips tool messages (planner has no tools bound).
- Invokes the base LLM.
- Parses the response through `_parse_planner_specialists`.
- Falls back to `["analyst"]` on any error or empty parse.
- Builds `stages_to_run = ["data_loader", "planner"] + selected + ["critic", "reporter"]`.

Why `FIXED_PREFIX` and `FIXED_SUFFIX`?

```
FIXED_PREFIX = ["data_loader", "planner"]
FIXED_SUFFIX = ["critic", "reporter"]
```

These are immutable. Every pipeline starts with data loading and planning, and ends with critic validation and report generation. Only the middle is dynamic.

_heuristic_check(messages, stage_name)

Type: Helper | Lines: 593–633

```
def _heuristic_check(messages: list[BaseMessage], stage_name: str) -> str | None:
    has_new_files = False
    has_error = False
    has_tool_output = False
    has_structured = False

    for msg in reversed(messages):
        msg_name = getattr(msg, "name", None)
        # Only inspect tool results from this specialist's iteration
        if isinstance(msg, ToolMessage) and msg.name == "execute_python":
            has_tool_output = True
            content = normalize_content(msg.content)
            if '"new_files":' in content:
                try:
                    import re as _re
                    match = _re.search(r'"new_files":\s*\[([^\]]+)\]', content)
                    if match and match.group(1).strip():
                        has_new_files = True
                except Exception:
                    pass
            if "STRUCTURED_OUTPUT:" in content:
                has_structured = True
            if '"error":' in content.lower() or "Traceback" in content or "Error:" in content:
                has_error = True
            # Stop scanning once we hit this specialist's first AI message
            if isinstance(msg, AIMessage) and msg_name == stage_name and not msg.tool_calls:
                break
            # Also stop if we hit a different stage's entry
            if isinstance(msg, AIMessage) and msg_name and msg_name != stage_name and msg_name != "manager":
                break

    if has_tool_output and has_new_files and has_structured and not has_error:
        return f"Heuristic auto-accept: {stage_name} produced files + STRUCTURED_OUTPUT, no errors."
    if has_tool_output and has_new_files and not has_error:
        return f"Heuristic auto-accept: {stage_name} produced files (no STRUCTURED_OUTPUT – degraded)."
    return None
```

The cost-saver. Every time a specialist finishes, the default next step is `manager_review`, which calls an LLM to judge quality. That's expensive. This function provides a **fast-path** that skips the LLM entirely when the output looks obviously good.

How the scan works:

It iterates **backwards** through the message list, looking at the most recent messages first. It stops when it hits:

1. This specialist's first non-tool-call `AIMessage` — the end of their current iteration.
2. A different stage's `AIMessage` — safety boundary to avoid inspecting stale messages.

What it looks for in `execute_python` outputs:

- **"new_files": [...]** — Did the specialist actually produce files (charts, reports)? The regex `r'"new_files":\s*\([^\)]+\)'` extracts the list and checks it is non-empty.
- **"STRUCTURED_OUTPUT:"** — Did the specialist emit structured data via the 2C protocol?
- **"error" / Traceback / Error:** — Was there any exception in the Python execution?

Decision tree:

1. **Strict auto-accept:** Produced files + structured output + no errors → instantly accept.
2. **Lenient auto-accept:** Produced files + no errors (but missing structured output) → accept with a degraded flag.
3. **No auto-accept:** Missing files, has errors, or ambiguous → fall back to LLM review.

Impact: In practice, this eliminates ~90% of manager LLM calls. When a specialist runs `execute_python`, produces charts, and exits cleanly, the graph instantly moves on.

`_maybe_extend_pipeline(state)`

Type: Helper | **Lines:** 635–663

```
def _maybe_extend_pipeline(state: AnalysisState) -> dict | None:
    ext_used = state.get("extensions_used", 0) or 0
    if ext_used >= _MAX_PIPELINE_EXTENSIONS:
        return None
    stages = list(state.get("stages_to_run", []))
    extras = _parse_extensions(state["messages"], stages)
    if not extras:
        return None
    try:
        reporter_idx = stages.index("reporter")
    except ValueError:
        reporter_idx = len(stages)
    new_stages = (
        stages[:reporter_idx]
        + extras
        + ["critic"]
    )
```

```

        + stages[reporter_idx:]
    )
    print(f" [manager] Critic extends pipeline with: {' '.join(extras)} "
          f"(extension {ext_used + 1}/{_MAX_PIPELINE_EXTENSIONS})")
    _progress("critic", f"Допълнителни специалисти: {' '.join(_label(s) for s in
extras)}")
    return {
        "stages_to_run": new_stages,
        "extensions_used": ext_used + 1,
    }

```

Adaptive pipeline extension. The critic can request additional specialists if it finds gaps in the analysis. This function splices them into the stage list.

Mechanics:

1. Check `extensions_used` against the cap (`_MAX_PIPELINE_EXTENSIONS = 2`).
2. Call `_parse_extensions` to find new specialist names from the critic's output.
3. Find the `"reporter"` index — new specialists are inserted **before** the reporter.
4. Insert `extras + ["critic"]` — a second critic run is added after the new specialists to review their work.
5. Return state update with new stage list and incremented extension counter.

Example: Original pipeline: `[data_loader, planner, analyst, critic, reporter]`
Critic requests `anomaly_detective`. New pipeline: `[data_loader, planner, analyst, anomaly_detective, critic, reporter]`.

manager_review_node(state)

Type: Node | **Lines:** 665–733

```

def manager_review_node(state: AnalysisState) -> dict:
    current = state.get("current_stage", "data_loader")
    attempts = state.get("stage_attempts", {})
    attempt_count = attempts.get(current, 0)

    print(f"\n [manager] Reviewing {current} output (attempt {attempt_count + 1})...")

    # Auto-accept infrastructure stages
    if current in _AUTO_ACCEPT_STAGES:
        print(f" [manager] {current} – auto-accepting (infrastructure stage).")
        update = {
            "messages": [AIMessage(content=f"ACCEPT: {current} completed.",
name="manager")],
            "stage_attempts": {**attempts, current: attempt_count + 1},
        }

```

```

        if current == "critic":
            ext_update = _maybe_extend_pipeline(state)
            if ext_update:
                update.update(ext_update)
            return update

    if attempt_count >= MAX_REWORKS_PER_STAGE:
        print(f" [manager] Max reworks reached for {current} – accepting.")
        return {
            "messages": [AIMessage(content=f"ACCEPT: Max reworks reached for {current}.",
name="manager")],
            "stage_attempts": {**attempts, current: attempt_count + 1},
        }

    # Heuristic fast-path
    heuristic_reason = _heuristic_check(state["messages"], current)
    if heuristic_reason:
        print(f" [manager] {heuristic_reason}")
        return {
            "messages": [AIMessage(content=f"ACCEPT: {heuristic_reason}",
name="manager")],
            "stage_attempts": {**attempts, current: attempt_count + 1},
        }

    # Fall back to LLM review
    print(f" [manager] Heuristic inconclusive for {current}, invoking LLM review...")
    compressed = compress_messages(state["messages"])
    messages = [SystemMessage(content=MANAGER_REVIEW_PROMPT)] +
strip_tool_messages(compressed)

    try:
        response = llm.invoke(messages)
    except Exception as e:
        print(f" [manager] Review error: {str(e)[:150]}")
        return {
            "messages": [AIMessage(content="ACCEPT: Review skipped due to error.",
name="manager")],
            "stage_attempts": {**attempts, current: attempt_count + 1},
        }

    raw = response.content
    content = normalize_content(raw).strip()

    if content.upper().startswith("REWORK") or "REWORK:" in content.upper():
        decision = "REWORK"
        stamped = f"REWORK: {content}"
    else:
        decision = "ACCEPT"
        stamped = f"ACCEPT: {content}"

    print(f" [manager] Decision: {decision} – {content[:150]}")
    if decision == "REWORK":
        _progress("manager", f"🔄 {_label(current)}: Необходима е корекция, повторен
опит...")

    return {
        "messages": [AIMessage(content=stamped, name="manager")],
        "stage_attempts": {**attempts, current: attempt_count + 1},
    }

```

The quality gate. Every stage except the terminal ones passes through here before advancing.

Decision hierarchy (in order):

1. **Infrastructure auto-accept** — `data_loader`, `planner`, `critic`, `reporter` are always accepted without inspection. For `critic`, it also checks `_maybe_extend_pipeline`.
2. **Max rework guardrail** — If `attempt_count >= 1`, forced accept. Prevents infinite loops.
3. **Heuristic fast-path** — If `_heuristic_check` returns a reason, instantly accept without LLM cost.
4. **LLM review fallback** — Compresses history, strips tool messages, sends to manager LLM with `MANAGER_REVIEW_PROMPT`. The LLM decides `ACCEPT` or `REWORK`.
5. **Error fallback** — If the LLM call itself fails, auto-accept to keep the pipeline moving.

The `stage_attempts` dict: Tracks how many times each stage has been reviewed. Keyed by stage name. On rework, the same stage runs again, and `attempt_count` increments. Once it hits the cap, the stage is forced through regardless of quality.

Stamping: The manager's response is prefixed with `ACCEPT:` or `REWORK:` so `manager_router` can parse it with a simple string check instead of fragile JSON parsing.

`tool_node(state)`

Type: Node (async) | **Lines:** 736–771

```
async def tool_node(state: AnalysisState) -> dict:
    last_message = state["messages"][-1]
    owner_name = getattr(last_message, "name", None) or "specialist"

    results = []
    for tc in last_message.tool_calls:
        tool = tools_by_name.get(tc["name"])
        if tool is None:
            results.append(ToolMessage(
                content=f"Error: unknown tool '{tc['name']}'",
                tool_call_id=tc["id"], name=tc["name"],
            ))
            continue
        try:
            print(f"    [tool] Executing {tc['name']}...")
            output = await tool.ainvoke(tc["args"])
            output_str = str(output)
            results.append(ToolMessage(
```



```

        content=output_str, tool_call_id=tc["id"], name=tc["name"],
    ))
    if tc["name"] == "execute_python":
        _stream_structured_outputs(owner_name, output_str)
except Exception as e:
    results.append(ToolMessage(
        content=f"Error: {e}", tool_call_id=tc["id"], name=tc["name"],
    ))
return {"messages": results}

```

The tool executor. This is where the graph actually *does* things — runs Python, queries the database, writes files.

Key details:

- **Owner tracking:** `owner_name` is extracted from the last message's `name` field. This is used to attribute streamed structured outputs to the correct specialist in the UI.
- **Tool lookup:** `tools_by_name` is a dict mapping tool names to tool objects. If a specialist hallucinates a non-existent tool, an error `ToolMessage` is returned instead of crashing.
- **Async execution:** `await tool.ainvoke(...)` allows I/O-bound tools (database queries, file writes) to run without blocking.
- **Structured output streaming:** Only for `execute_python` — after the tool returns, `_stream_structured_outputs` scans the output for `STRUCTURED_OUTPUT:` lines and forwards them as progress events.
- **Error isolation:** Each tool call is wrapped in its own `try/except`. If one tool fails, the others still run. The error is returned as a `ToolMessage` so the specialist can see it and decide what to do.

Returns: `{"messages": results}` — a list of `ToolMessages`, one per tool call.

specialist_router(state)

Type: Router | **Lines:** 775–789

```

def specialist_router(state: AnalysisState) -> str:
    last = state["messages"][-1]
    if hasattr(last, "tool_calls") and last.tool_calls:
        return "tools"
    if state.get("parallel_mode"):
        return "critic_entry"
    stages = state.get("stages_to_run") or []
    current = state.get("current_stage")

```

```
if stages and current == stages[-1]:
    return "end"
return "manager_review"
```

The specialist's traffic cop. After a specialist node finishes, this router decides where to go next. It handles four distinct cases:

1. **"tools"** — The specialist's last message contains **tool_calls**. It wants to execute tools, so route to the **tool_node**.
2. **"critic_entry"** — In parallel mode, all specialists converge at the critic after finishing. This skips individual manager review for each specialist.
3. **"end"** — If this is the last stage in the pipeline (the **reporter**), skip the manager review and go straight to **END**.
4. **"manager_review"** — Normal sequential path. The specialist finished without tool calls and is not the terminal stage, so go to quality review.

Why check **tool_calls first?** A specialist might finish its text response but also include tool calls in the same message. LangGraph requires us to route to **tools** before anything else, because the tools must be executed before the specialist can truly be considered "done."

after_tools(state)

Type: Router | **Lines:** 791–795

```
def after_tools(state: AnalysisState) -> str:
    for msg in reversed(state["messages"]):
        if hasattr(msg, "tool_calls") and msg.tool_calls and getattr(msg, "name", None):
            return msg.name
    return "data_loader"
```

Routes **tool_node back to whoever requested the tools.** After tools execute, we need to return the results to the specialist that asked for them.

How it works:

Scans **backwards** through the message history looking for the most recent **AIMessage** that has **tool_calls** and a **name**. That **name** is the specialist that needs the results.

Why backwards? In a sequential pipeline, the requesting specialist is always the most recent one with tool calls. Scanning backwards finds it in $O(1)$ average time.

Fallback: "data_loader" — should never happen in practice, but provides a safe default.

_parse_extensions(messages, stages)

Type: Helper | **Lines:** 800–816

```
def _parse_extensions(messages: list[BaseMessage], stages: list[str]) -> list[str]:
    for msg in reversed(messages):
        if isinstance(msg, AIMessage) and getattr(msg, "name", None) == "critic":
            text = normalize_content(msg.content)
            for line in text.splitlines():
                line = line.strip()
                if line.upper().startswith("EXTEND_PIPELINE:"):
                    raw = line.split(":", 1)[1].strip()
                    candidates = [s.strip().lower().replace(" ", "_") for s in
raw.split(",")]

                    return [
                        s for s in candidates
                        if s in SPECIALIST_POOL and s not in stages
                   ][:2]
            return []
    return []
```

Finds critic-requested pipeline extensions. The critic can output a line like:

```
EXTEND_PIPELINE: forecaster, optimizer
```

Parsing rules:

1. Scan backwards for the most recent critic message.
2. Split into lines, find the one starting with **EXTEND_PIPELINE:**.
3. Extract after the colon, split on commas.
4. Normalize each candidate (lowercase, strip spaces → underscores).
5. **Filter:** Must be in **SPECIALIST_POOL** AND not already in current **stages**.
6. **Cap:** Return at most 2 new specialists.

Why filter against existing stages? Prevents the critic from requesting a specialist that is already scheduled to run.

Why cap at 2? Limits pipeline bloat. The critic is asked to be selective.

manager_router(state)

Type: Router | Lines: 818–855

```
def manager_router(state: AnalysisState):
    last = state["messages"][-1]
    content = last.content if isinstance(last.content, str) else str(last.content)
    current = state.get("current_stage", "data_loader")
    stages = list(state.get("stages_to_run", FIXED_PREFIX + ["analyst"] + FIXED_SUFFIX))

    # REWORK → send back to current stage
    if content.startswith("REWORK:"):
        print(f" [manager] Sending {current} back for rework.")
        return f"{current}_entry"

    # PARALLEL FAN-OUT: planner just accepted → dispatch all specialists at once
    if _ENABLE_PARALLEL and current == "planner":
        specialists = [s for s in stages if s in SPECIALIST_POOL]
        if specialists:
            print(f"\n → Parallel fan-out: {'', '.join(specialists)}")
            _progress("planner", f"Паралелна обработка: {'', '.join(_label(s) for s in
specialists)}")
            base = dict(state)
            base["parallel_mode"] = True
            return [Send(f"{s}_entry", base) for s in specialists]
        return "critic_entry"

    # ACCEPT → advance to next stage (sequential path)
    if current in stages:
        idx = stages.index(current)
        if idx + 1 < len(stages):
            next_stage = stages[idx + 1]
            print(f"\n → Advancing: {current} → {next_stage}")
            return f"{next_stage}_entry"

    print(f"\n → Pipeline complete!")
    return "end"
```

The most powerful router in the graph. It can return a string, or a list of **Send** objects (LangGraph's mechanism for spawning parallel branches).

Three paths:

- 1. REWORK (lines 828–830)** If the manager's message starts with **"REWORK:"**, route back to **{current}_entry**. The same specialist runs again. This is how the rework loop works.
- 2. Parallel fan-out (lines 833–841)** If parallel mode is enabled AND the current stage is **"planner"**, this is the magic moment. Instead of routing to the next stage sequentially, it:

- Extracts all specialist names from `stages`.
- Creates a **copy** of the current state with `parallel_mode = True`.
- Returns a list of `Send(f"{specialist}_entry", state_copy)` — one per specialist.

LangGraph receives this list and spawns independent branches for each specialist. Each branch gets its own state copy, so they don't interfere with each other.

3. Sequential accept (lines 847–852) Normal case: find `current` in `stages`, get the next stage, route to its entry node.

4. Complete (lines 854–855) If `current` is not found or at the end, route to `"end"` which maps to LangGraph `END`.

`make_stage_entry(stage_name) → entry_node`

Type: Factory | **Lines:** 858–866

```
def make_stage_entry(stage_name: str):
    def entry_node(state: AnalysisState) -> dict:
        update: dict = {"current_stage": stage_name}
        if stage_name == "critic":
            update["parallel_mode"] = False
        return update
    return entry_node
```

The stage bookmarker. Entry nodes exist because LangGraph's `add_edge` requires a source node. We cannot directly route to `analyst` from `manager_router`; we must route to `analyst_entry` first, which updates `current_stage` in state, then the unconditional edge from `analyst_entry` → `analyst` takes over.

Special case for `critic`:

```
if stage_name == "critic":
    update["parallel_mode"] = False
```

When parallel branches converge at `critic_entry`, they need to switch back to sequential mode. Otherwise, the `specialist_router` would route the critic itself to `"critic_entry"` again (because `parallel_mode` would still be `True`), creating an infinite loop.

Graph Assembly (lines 869–924)

```
graph = StateGraph(AnalysisState)

# Register entry + specialist nodes for ALL possible stages
for stage in ALL_STAGES:
    graph.add_node(f"{stage}_entry", make_stage_entry(stage))
    graph.add_node(stage, make_specialist_node(stage))

# Planner is special – not a specialist, no tools
graph.add_node("planner_entry", make_stage_entry("planner"))
graph.add_node("planner", planner_node)

graph.add_node("tools", tool_node)
graph.add_node("manager_review", manager_review_node)

# Entry point
graph.set_entry_point("data_loader_entry")

# Wire: entry → node (for all stages + planner)
for stage in ALL_STAGES:
    graph.add_edge(f"{stage}_entry", stage)
graph.add_edge("planner_entry", "planner")

# Wire: planner → manager_review (planner doesn't use tools)
graph.add_edge("planner", "manager_review")

# Wire: specialist → tools / manager_review / critic_entry / END
for stage in ALL_STAGES:
    graph.add_conditional_edges(
        stage,
        specialist_router,
        {
            "tools": "tools",
            "manager_review": "manager_review",
            "critic_entry": "critic_entry",
            "end": END,
        },
    )

# Wire: tools → back to specialist
graph.add_conditional_edges(
    "tools",
    after_tools,
    {stage: stage for stage in ALL_STAGES},
)

# Wire: manager_review → next stage or rework or END
manager_targets = {f"{stage}_entry": f"{stage}_entry" for stage in ALL_STAGES}
manager_targets["planner_entry"] = "planner_entry"
manager_targets["end"] = END
graph.add_conditional_edges(
    "manager_review",
    manager_router,
    manager_targets,
```

```
)  
  
return graph.compile(checkpointer=checkpointer)
```

The wiring diagram. Every node and edge is explicitly declared. There are no implicit routes.

Key design decisions:

- 1. Entry nodes for every stage** Even though `data_loader` could theoretically be the entry point directly, we use `data_loader_entry` → `data_loader` so that `manager_router` can route to `{stage}_entry` and the stage name gets updated in state before the specialist runs.
- 2. Planner is special** Planner is not created by `make_specialist_node` because it has no tools and uses `planner_node` directly. It also has a direct edge to `manager_review` (no `specialist_router`) because it never calls tools.
- 3. `add_conditional_edges` maps string keys to node names** The router functions return string keys like `"tools"` or `"manager_review"`. LangGraph looks up these keys in the mapping dict to decide which node to visit next.
- 4. `checkpointer`** Passed to `graph.compile()` for persistence. If the process crashes mid-analysis, LangGraph can resume from the last checkpoint.

Part 3: `build_followup_graph` — Follow-Up Builder

`_split_leading_system(msgs)`

Type: Helper | **Lines:** 1031–1037

```
def _split_leading_system(msgs: list[BaseMessage]) -> tuple[list[BaseMessage],  
list[BaseMessage]]:  
    leading: list[BaseMessage] = []  
    i = 0  
    while i < len(msgs) and isinstance(msgs[i], SystemMessage):  
        leading.append(msgs[i])  
        i += 1  
    return leading, msgs[i:]
```

Context-preserving truncation. The follow-up graph takes the last N messages to keep token usage low. But if the API layer injected a `SystemMessage` containing the current report snapshot, we must **never** drop it. This function separates leading system messages from the rest so the tail-slice logic can safely discard old messages while keeping system context.

followup_router_node(state)

Type: Node | Lines: 1040–1126

```
def followup_router_node(state: FollowUpState) -> dict:
    _progress("followup", "Анализиране на допълнителния въпрос...")
    leading_sys, rest = _split_leading_system(state["messages"])
    messages = [SystemMessage(content=FOLLOWUP_ROUTER_PROMPT)] + leading_sys + rest[-20:]

    try:
        response = llm.invoke(messages)
    except Exception as e:
        print(f" [followup_router] Error: {str(e)[:150]}")
        return {
            "messages": [AIMessage(content="ACTION: ANSWER\nINSTRUCTION: Answer the
user's question directly.", name="followup_router")],
            "action": "ANSWER",
            "instruction": "Answer the user's question directly.",
        }

    raw = response.content
    if isinstance(raw, list):
        content = "\n".join(
            item.get("text", "") if isinstance(item, dict) else str(item)
            for item in raw
        ).strip()
    else:
        content = (str(raw) if raw else "").strip()

    # Parse action / instruction
    action = "ANSWER"
    instruction = "Answer the user's question."
    for line in content.split("\n"):
        line = line.strip()
        if line.upper().startswith("ACTION:"):
            action = line.split(":", 1)[1].strip()
        elif line.upper().startswith("INSTRUCTION:"):
            instruction = line.split(":", 1)[1].strip()

    # SAFETY NET – override ANSWER if chart keywords detected
    last_human = next(
        (m for m in reversed(state["messages"]) if isinstance(m, HumanMessage)),
        None,
    )
    user_text = str(last_human.content).lower() if last_human else ""
    CHART_KEYWORDS = (
        "начертай", "изготви", "генерирай", "покажи график",
        "графика", "графики", "хистограм", "визуализирай",
```



```

        "диаграм", "plot", "draw", "chart", "figure", "histogram",
        "visuali", "forecast", "прогноз",
    )
    if action.upper() == "ANSWER" and any(k in user_text for k in CHART_KEYWORDS):
        action = "SPECIALIST:analyst"
        instruction = "Създай поискните графики/визуализации..."

    _progress("followup", f"Действие: {action}")
    marker = f"[router] action={action} | instruction={instruction[:160]}"
    return {
        "messages": [AIMessage(content=marker, name="followup_router")],
        "action": action,
        "instruction": instruction,
    }

```

The follow-up decision maker. After the main analysis is complete, the user can ask follow-up questions. This node decides what to do without re-running the entire pipeline.

Parsing the LLM response:

The router LLM is instructed to output exactly two lines:

```

ACTION: SPECIALIST:analyst
INSTRUCTION: Create a correlation heatmap for mill 8

```

We scan all lines for **ACTION:** and **INSTRUCTION:** prefixes. The parsing is lenient — the LLM may add prose before or after, and we still extract the structured lines.

The safety net (lines 1084–1115):

The most important reliability feature. LLMs often ignore the **ACTION:** contract and just answer directly. If the user says "начертай графика" (draw a chart) but the router chose **ANSWER**, we override it to **SPECIALIST:analyst**. This prevents the common failure mode where the user asks for a new chart but gets a text answer instead.

Why store a marker?

Instead of storing the raw LLM output (which may contain hallucinations) in state, we store a short deterministic string: `[router] action=SPECIALIST:analyst | instruction=...`. This keeps the state clean and prevents the router's verbose output from leaking into downstream prompts.

followup_executor_node(state)

Type: Node | Lines: 1129–1189

```
def followup_executor_node(state: FollowUpState) -> dict:
    action = state.get("action", "ANSWER")
    instruction = state.get("instruction", "")
    _progress("followup", "Изпълнение на допълнителен анализ...")

    if action.upper().startswith("SPECIALIST:"):
        specialist_name = action.split(":", 1)[1].strip().lower()
        system = f"{FOLLOWUP_SPECIALIST_PROMPT}\n\nYou are acting as the
{specialist_name} specialist.\nTask: {instruction}"
    elif action.upper() == "REFINE_REPORT":
        system = f"{FOLLOWUP_SPECIALIST_PROMPT}\n\nTask: Refine/update the existing
report. {instruction}..."
    else:
        system = f"{FOLLOWUP_SPECIALIST_PROMPT}\n\nTask: {instruction}\nAnswer the user's
question..."

    leading_sys, rest = _split_leading_system(state["messages"])
    focused = leading_sys + rest[-30:]
    messages = [SystemMessage(content=system)] + focused

    try:
        response = followup_llm.invoke(messages)
    except Exception as e:
        return {
            "messages": [AIMessage(content=f"Error during follow-up: {str(e)[:200]}"),
name="followup_executor")],
        }

    response.name = "followup_executor"
    return {"messages": [response]}
```

The follow-up worker. This is a single node that handles all three follow-up action types by building different system prompts.

Three branches:

1. SPECIALIST (lines 1135–1138) Injects `FOLLOWUP_SPECIALIST_PROMPT` + a role identity line. The LLM knows it is acting as a specific specialist (e.g. `analyst`) and should use tools to produce new charts or computations.

2. REFINE_REPORT (lines 1139–1146) Instructs the LLM to update the existing Markdown report without running new analysis. It typically calls `list_output_files` and `write_markdown_report`.

3. ANSWER (lines 1147–1162) Direct text answer using already-computed results. Includes strict anti-hallucination rules:

- Do not claim files exist unless confirmed via `list_output_files`.
- If the user wants something new, call the tool first, wait for the result, then announce success.
- If unable to execute, say clearly what is missing instead of fabricating an answer.

Message window: Last 30 messages + preserved leading system messages. This is larger than the router's window because the executor needs more context to actually perform the work.

followup_tool_node(state)

Type: Node (async) | **Lines:** 1192–1213

Identical logic to the main graph's `tool_node`, but for the follow-up graph. Iterates `last_message.tool_calls`, executes each with `await tool.ainvoke(...)`, catches exceptions, returns `ToolMessages`.

Key difference: It does **not** call `_stream_structured_outputs`. The follow-up graph does not stream structured data to the frontend; it only produces the final result.

executor_router(state)

Type: Router | **Lines:** 1216–1220

```
def executor_router(state: FollowUpState) -> str:
    last = state["messages"][-1]
    if hasattr(last, "tool_calls") and last.tool_calls:
        return "followup_tools"
    return "end"
```

Even simpler than the main graph's router. Only two destinations:

- `"followup_tools"` — executor requested tools, go run them.
 - `"end"` — executor is done, finish the follow-up.
-

Follow-Up Graph Assembly (lines 1223–1238)

```
graph = StateGraph(FollowUpState)

graph.add_node("followup_router", followup_router_node)
graph.add_node("followup_executor", followup_executor_node)
graph.add_node("followup_tools", followup_tool_node)

graph.set_entry_point("followup_router")
graph.add_edge("followup_router", "followup_executor")
graph.add_conditional_edges(
    "followup_executor",
    executor_router,
    {"followup_tools": "followup_tools", "end": END},
)
graph.add_edge("followup_tools", "followup_executor")

return graph.compile(checkpointer=checkpointer)
```

Much simpler than the main graph. Only 3 nodes and a simple loop. The follow-up graph reuses the same tool registry but has no planner, no manager, no critic, and no parallel fan-out.

Part 4: Logic Flows

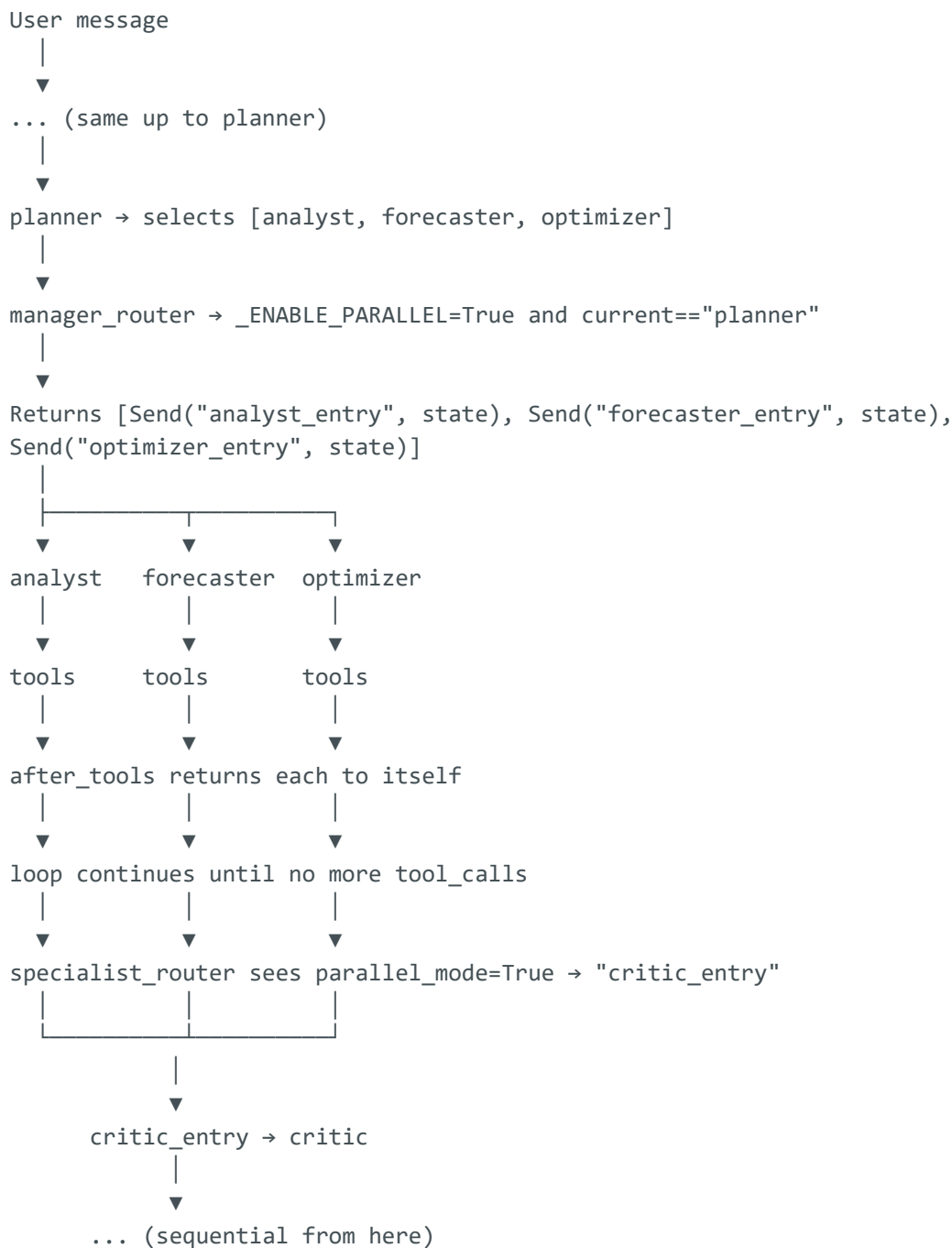
Flow 1: New Question → Sequential Analysis

```
graph TD
    A[User message] --> B[Frontend → POST /api/v1/agent/analyze]
    B --> C["api_endpoint.py → build_graph(tools, api_key, on_progress, ...)"]
    C --> D[Graph entry: data_loader_entry]
    D --> E["data_loader → query_mill_data / query_combined_data → ToolMessages"]
    E --> F[ ]
```



▼
LangGraph END → frontend poll returns completed status + file list

Flow 2: New Question → Parallel Analysis



Key difference: In parallel mode, all three specialists run concurrently. Their states are independent copies. They converge at **critic_entry** because **specialist_router** routes to **"critic_entry"** when **parallel_mode=True**.

Flow 3: Manager Rework Loop

```
specialist_X finishes → specialist_router → "manager_review"
|
▼
manager_review_node:
- attempt_count = 0
- heuristic inconclusive
- LLM review returns "REWORK: needs better correlation chart"
|
▼
manager_router sees "REWORK:" → "specialist_X_entry"
|
▼
specialist_X runs again (iteration 2)
|
▼
manager_review_node:
- attempt_count = 1 (MAX_REWORKS reached)
- forced ACCEPT regardless of output quality
|
▼
manager_router → next stage
```

Guardrail: Max 1 rework per stage. After that, the graph moves forward even if the output is imperfect. Prevents infinite loops.

Flow 4: Critic Pipeline Extension

```
... (analyst and forecaster have finished)
|
▼
critic runs, finds missing anomaly analysis
|
▼
critic outputs "EXTEND_PIPELINE: anomaly_detective, bayesian_analyst"
|
▼
manager_review_node (current == "critic"):
- auto-accepts (infrastructure stage)
- calls _maybe_extend_pipeline
- extensions_used = 0 < 2 → proceed
|
▼
_parse_extensions finds ["anomaly_detective", "bayesian_analyst"]
|
▼
New stages spliced: [..., "anomaly_detective", "bayesian_analyst", "critic", "reporter"]
|
```

```
▼
manager_router → "anomaly_detective_entry"
|
▼
... (new specialists run, then second critic reviews them)
|
▼
reporter → final report
```

Limit: Max 2 extensions. The second critic can also request extensions, but `_maybe_extend_pipeline` will reject it if the cap is reached.

Flow 5: Follow-Up Question

```
User replies to completed report
|
▼
Frontend → POST /api/v1/agentic/analyze (with prior analysis ID)
|
▼
api_endpoint.py → build_followup_graph(...)
|
▼
Entry: followup_router
|
▼
LLM decides action = "SPECIALIST:analyst"
|
▼
followup_router → followup_executor
|
▼
LLM with tools bound → calls execute_python → creates new charts
|
▼
executor_router → has tool_calls → "followup_tools"
|
▼
tool_node runs Python → returns results
|
▼
executor_router → no more tool_calls → "end"
|
▼
Frontend poll returns updated files
```

Alternative paths:

- `REFINE_REPORT` → executor calls `write_markdown_report` with updated text.
- `ANSWER` → executor replies directly with text, no tool calls.

Flow 6: Heuristic Auto-Accept (Happy Path)

```
analyst → execute_python
|
▼
Tool output contains:
- "new_files": ["chart1.png", "chart2.png"]
- "STRUCTURED_OUTPUT:{...}"
- No errors
|
▼
specialist_router → no more tool_calls → manager_review
|
▼
manager_review_node:
- _heuristic_check scans backwards
- has_new_files=True, has_structured=True, has_error=False
- returns "Heuristic auto-accept: analyst produced files + STRUCTURED_OUTPUT, no
errors."
|
▼
No LLM call made → instant ACCEPT
|
▼
manager_router → next stage
```

Performance impact: Saves ~1 LLM call per specialist in the common case (produced charts successfully). Significant cost reduction.

Part 5: Configuration Overrides

At the top of `build_graph`, module-level constants are shadowed by a per-request `settings` dict:

```
def build_graph(..., settings: dict | None = None, ...):
    _settings = settings or {}
    _MAX_TOOL_OUTPUT_CHARS = _settings.get("maxToolOutputChars", MAX_TOOL_OUTPUT_CHARS)
    _MAX_AI_MSG_CHARS = _settings.get("maxAiMessageChars", MAX_AI_MSG_CHARS)
    _MAX_MESSAGES_WINDOW = _settings.get("maxMessagesWindow", MAX_MESSAGES_WINDOW)
    _MAX_SPECIALIST_ITERS = _settings.get("maxSpecialistIterations",
MAX_SPECIALIST_ITERS)
    _ENABLE_PARALLEL = bool(_settings.get("enableParallelSpecialists", False))
    _reporter_model = _settings.get("reporterModel") or GEMINI_REPORTER_MODEL
```

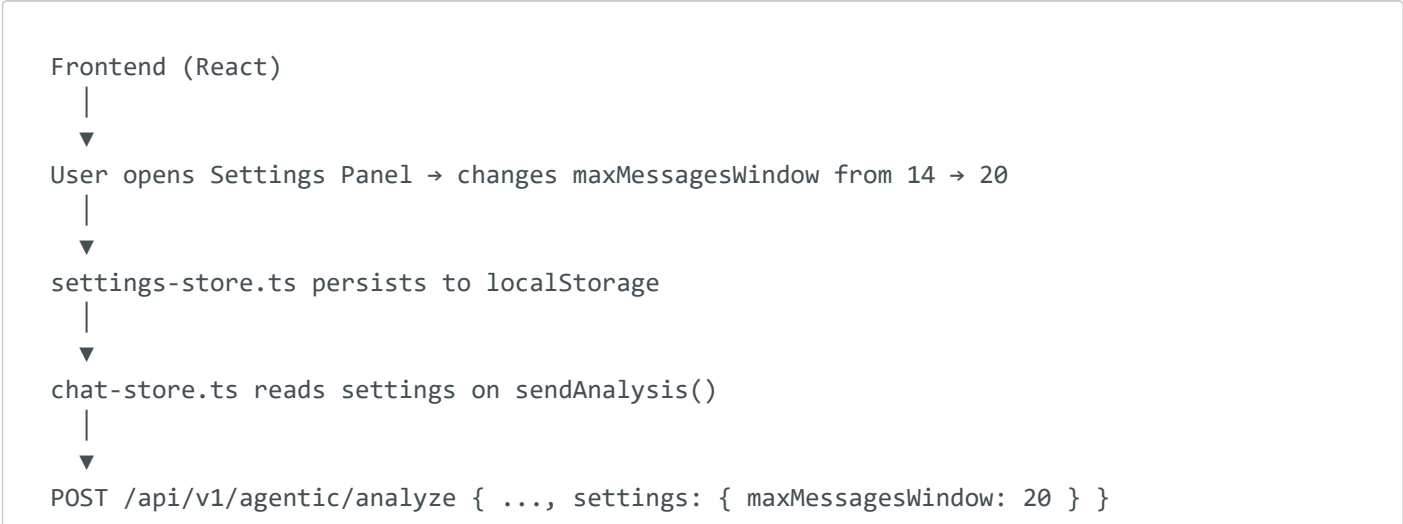
This means the same running Python process can handle two concurrent analyses with **different** token budgets, because each call to `build_graph` creates its own closure with local `_MAX_*` variables.

Settings table

Setting	Default	Affected function(s)	What happens when you increase it
<code>maxToolOutputChars</code>	2000	<code>compress_messages</code> , <code>build_focused_context</code>	Tool outputs (especially <code>execute_python</code> stdout) are truncated less aggressively. Specialists see more of their prior tool results.
<code>maxAiMessageChars</code>	3000	<code>compress_messages</code>	AI messages are kept longer. Useful if a specialist writes very long prose responses.
<code>maxMessagesWindow</code>	14	<code>compress_messages</code>	More recent messages are kept in the sliding window. Increases context accuracy but also token usage.
<code>maxSpecialistIterations</code>	5	<code>specialist_node</code>	A specialist can loop more times (call tools, get results, think again). Risk of

Setting	Default	Affected function(s)	What happens when you increase it
			infinite loops if set too high.
<code>enableParallelSpecialists</code>	False	<code>manager_router</code>	When True, the planner's selected specialists run concurrently instead of sequentially. Faster wall-clock time but higher peak token usage.
<code>reporterModel</code>	<code>GEMINI_MODEL</code>	Reporter node LLM init	Allows using a stronger (more expensive) model for the final report synthesis while keeping the rest of the pipeline on a cheaper model.

Override path through the stack:



|
▼
api_endpoint.py: AnalysisRequest model validates settings
|
▼
build_graph(..., settings={"maxMessagesWindow": 20})
|
▼
Local _MAX_MESSAGES_WINDOW = 20 shadows module-level MAX_MESSAGES_WINDOW (14)
|
▼
All nodes inside this graph closure use _MAX_MESSAGES_WINDOW

Important: These are *per-request* overrides. They do not mutate global state. If two users send analyses simultaneously, one with default settings and one with custom settings, each gets its own `build_graph` closure with its own values.